Mauricio de Souza Coutinho

Exercício 1:

Liste pedidos com nome do cliente e status:

```
73 •  SELECT p.id_pedido, c.nome_cliente, p.status_pedido
74    FROM PEDIDO p
75    INNER JOIN CLIENTE c ON p.id_cliente = c.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Con

id_pedido	nome_cliente	status_pedido
1	Ana Silva	Em Andamento
2	Carlos Souza	Concluído

Liste os produtos e subtotais de cada item:

```
77 •  SELECT pr.nome_produto, ip.subtotal
78    FROM ITEM_PEDIDO ip
79    INNER JOIN PRODUTO pr ON ip.id_produto = pr.id_produto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nome_produto	subtotal
Teclado Mecânico	150.00
Mouse Gamer	161.00

Monte o caminho PEDIDO , ITEM_PEDIDO e PRODUTO:

Montar o caminho = Juntar as 3 tabelas.

```
81    -- p = pedido / pr = produto / ip = item_pedido
82 •  SELECT p.id_pedido , pr.nome_produto , ip.quantidade , ip.subtotal
83    FROM PEDIDO p
84    INNER JOIN ITEM_PEDIDO ip ON p.id_pedido = ip.id_pedido
85    INNER JOIN PRODUTO pr ON ip.id_produto = pr.id_produto
86
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	nome_produto	quantidade	subtotal
1	Teclado Mecânico	1	150.00
2	Mouse Gamer	2	161.00

Liste empregados e respectivos supervisores:

```
87      -- e = empregado / s = supervisor
88      SELECT e.nome_empregado AS Empregado , s.nome_empregado AS Supervisor
89      FROM EMPREGADO e
90      INNER JOIN EMPREGADO s ON e.id_supervisor = s.id_empregado;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Empregado	Supervisor
Maria Funcionaria	João Gerente

Explique por que cada condição ON preserva a semântica:

Nas junções de tabelas distintas (Ex: **Pedido e Cliente**): A condição ON (p.id_cliente = c.id_cliente) preserva a semântica de negócio porque garante que os dados combinados pertençam estritamente ao **MESMO** cliente, evitando cruzamentos incorretos (produto cartesiano).

No autorrelacionamento (Ex: Empregado e Supervisor): A condição ON (e.id_supervisor = s.id_empregado) preserva a **lógica hierárquica**, assegurando que o registro da tabela à direita (s) atue exatamente como o chefe/gerente daquele funcionário específico da tabela à esquerda (e).

Exercício 2:

Liste todos os Clientes , inclusive os que não possuem pedidos:

Para garantir que todos os clientes apareçam na listagem — mesmo aqueles que nunca fizeram nenhum pedido —, utilizamos um LEFT JOIN tendo a tabela CLIENTE como base:

```
92      SELECT c.id_cliente, c.nome_cliente, p.id_pedido, p.status_pedido
93      FROM CLIENTE c
94      LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome_cliente	id_pedido	status_pedido
1	Ana Silva	1	Em Andamento
2	Carlos Souza	2	Concluído

Localize produtos nunca vendidos:

```sql
SELECT pr.id_produto, pr.nome_produto
FROM PRODUTO pr
LEFT JOIN ITEM_PEDIDO ip ON pr.id_produto = ip.id_produto
WHERE ip.id_item IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_produto	nome_produto
3	Monitor Ultrawide
4	Webcam Full HD

Reescreva um RIGHT JOIN equivalente com um LEFT JOIN:

```sql
SELECT c.nome_cliente, p.id_pedido
FROM CLIENTE c
RIGHT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nome_cliente	id_pedido
Ana Silva	1
Carlos Souza	2

```sql
SELECT c.nome_cliente, p.id_pedido
FROM PEDIDO p
LEFT JOIN CLIENTE c ON c.id_cliente = p.id_cliente
```

Result Grid | Filter Rows: | Export: | Wrap Cell Con

nome_cliente	id_pedido
Ana Silva	1
Carlos Souza	2

Simule uma junção externa completa em duas tabelas:

```sql
SELECT c.id_cliente, p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
UNION
SELECT c.id_cliente, p.id_pedido
FROM CLIENTE c
RIGHT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

id_cliente	id_pedido
1	1
2	2
3	NULL
4	NULL

Explique por que um filtro no WHERE pode eliminar linhas preservadas:

Explicação: Numa junção externa (**LEFT JOIN ou RIGHT JOIN**), as linhas sem correspondência recebem valores **NULL** nas colunas da tabela oposta. Se colocarmos uma condição restritiva na cláusula **WHERE** sobre uma coluna da tabela oposta (por exemplo, **WHERE p.status_pedido = 'Concluído'**), o valor **NULL** gerado para os registos sem correspondência falhará no teste lógico do filtro. Como resultado, essas linhas preservadas são eliminadas do resultado final, fazendo com que a consulta se comporte incorretamente como um **INNER JOIN**.

Boa prática: Condições específicas da tabela secundária que não devem eliminar as linhas preservadas devem ser colocadas diretamente na cláusula ON ou tratadas verificando explicitamente a nulidade (OR ... IS NULL).

Exercício 3:

Calcule quantidade , menor , maior e média de preços por categoria:

```
129 •   SELECT
130         categoria_produto,
131         COUNT(*) AS quantidade_produtos,
132         MIN(preco_produto) AS menor_preco,
133         MAX(preco_produto) AS maior_preco,
134         AVG(preco_produto) AS media_preco
135     FROM PRODUTO
136     GROUP BY categoria_produto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

categoria_produto	quantidade_produtos	menor_preco	maior_preco	media_preco
NULL	4	80.50	1200.00	420.125000

Conte pedidos por cliente , incluindo zero:

```
138 •   SELECT
139         c.id_cliente,
140         c.nome_cliente,
141         COUNT(p.id_pedido) AS total_pedidos
142     FROM CLIENTE c
143     LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
144     GROUP BY c.id_cliente, c.nome_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

id_cliente	nome_cliente	total_pedidos
1	Ana Silva	1
2	Carlos Souza	1
3	Fernanda Lima	0
4	Lucas Mendes	0

Mostre apenas clientes com pelo menos dois pedidos:

```
193 •  SELECT c.id_cliente, c.nome_cliente, COUNT(p.id_pedido) AS pedidos_concluidos
194    FROM CLIENTE c
195    INNER JOIN PEDIDO p ON c.id_cliente = p.id_cliente
196    WHERE p.status_pedido = 'Concluído'
197    GROUP BY c.id_cliente, c.nome_cliente
198    HAVING COUNT(p.id_pedido) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome_cliente	pedidos_concluidos

Calcule valor total por pedido usando ITEM_PEDIDO:

```
155 •  SELECT
156        id_pedido,
157        SUM(subtotal) AS valor_total_pedido
158    FROM ITEM_PEDIDO
159    GROUP BY id_pedido;
```

Result Grid | Filter Rows: | Export:

id_pedido	valor_total_pedido
1	150.00
2	161.00

Justifique cada uso de WHERE , GROUP BY e HAVING:

WHERE: Utilizado para filtrar as linhas individuais da tabela antes de qualquer agregação ou agrupamento. Reduz o volume de dados processados com base em condições específicas de linhas.

GROUP BY: Utilizado para agrupar as linhas que partilham os mesmos valores em colunas específicas, permitindo a aplicação de funções de agregação (como COUNT, SUM, AVG) por cada grupo formado.

HAVING: Utilizado para filtrar os grupos gerados pelo GROUP BY após as agregações terem sido calculadas. É indispensável quando precisamos aplicar condições sobre o resultado de funções agregadas (como COUNT(...) >= 2).

Exercício Integrador:

Duas consultas com INNER JOIN e qualificação completa:

Pergunta 1 : Quais são os nomes dos clientes e os respetivos estados de todos os pedidos registados?

```
161 •   SELECT c.nome_cliente, p.status_pedido
162     FROM CLIENTE c
163     INNER JOIN PEDIDO p ON c.id_cliente = p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

nome_cliente	status_pedido
Ana Silva	Em Andamento
Carlos Souza	Concluído

Expectativa: Retornar uma listagem com o nome do cliente emparelhado diretamente ao status de cada pedido que lhe pertence.

Interpretação: A qualificação completa através de aliases (c e p) evita ambiguidades e garante que apenas pedidos associados a clientes válidos sejam exibidos.

Pergunta 2: Qual é a relação detalhada dos produtos comprados em cada item de pedido, incluindo quantidade e subtotal?

```
165 •   SELECT p.nome_produto, ip.quantidade, ip.subtotal
166     FROM ITEM_PEDIDO ip
167     INNER JOIN PRODUTO p ON ip.id_produto = p.id_produto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

nome_produto	quantidade	subtotal
Teclado Mecânico	1	150.00
Mouse Gamer	2	161.00

Expectativa: Apresentar o nome comercial de cada produto associado às respetivas linhas de itens de pedidos.

Interpretação: Faz a ponte relacional entre o catálogo de produtos e as transações de venda registadas.

Duas consultas externas (RIGHT JOIN , LEFT JOIN) incluindo uma busca por ausência:

Pergunta 1: É possível listar todos os clientes do sistema juntamente com os seus pedidos, independentemente de terem ou não efetuado compras?

```
169 •  SELECT c.nome_cliente, p.id_pedido, p.status_pedido
170    FROM CLIENTE c
171    LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

nome_cliente	id_pedido	status_pedido
Ana Silva	1	Em Andamento
Carlos Souza	2	Concluído
Fernanda Lima	NULL	NULL
Lucas Mendes	NULL	NULL

Expectativa: Mostrar todos os clientes; aqueles sem compras apresentarão valores NULL nos campos de pedido.

Interpretação: O LEFT JOIN preserva todos os registos da tabela primária (CLIENTE) mesmo na ausência de correspondência à direita.

Pergunta 2: Quais são os produtos do nosso catálogo que nunca foram vendidos a nenhum cliente?

```
173 •  SELECT p.id_produto, p.nome_produto
174    FROM PRODUTO p
175    LEFT JOIN ITEM_PEDIDO ip ON p.id_produto = ip.id_produto
176    WHERE ip.id_item IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_produto	nome_produto
3	Monitor Ultrawide
4	Webcam Full HD

Expectativa: Isolar unicamente os produtos que não possuem nenhuma referência na tabela de itens de pedido.

Interpretação: A combinação de LEFT JOIN com o filtro WHERE ... IS NULL é o método padrão em SQL para detetar registos órfãos ou ausentes de correspondência.

Uma consulta com autorrelacionamento:

Pergunta: Como estruturar a listagem que exibe cada empregado ao lado do respetivo supervisor hierárquico?

```
178 •  SELECT e.nome_empregado AS Empregado, s.nome_empregado AS Supervisor
179    FROM EMPREGADO e
180    INNER JOIN EMPREGADO s ON e.id_supervisor = s.id_empregado;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Empregado	Supervisor
Maria Funcionaria	João Gerente

Expectativa: Relacionar colaboradores com os respetivos chefes dentro da mesma estrutura de tabela.

I**nterpretação:** O uso de dois aliases para a mesma tabela (e e s) simula duas entidades distintas, permitindo mapear a árvore hierárquica interna da organização.

Duas consultas agregadas com LEFT JOIN e contagem correta de zero:

Pergunta 1: Qual é a contagem exata de pedidos efetuados por cada cliente, assegurando que os clientes sem compras registem o valor zero?

```
182 •  SELECT c.id_cliente, c.nome_cliente, COUNT(p.id_pedido) AS total_pedidos
183    FROM CLIENTE c
184    LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
185    GROUP BY c.id_cliente, c.nome_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome_cliente	total_pedidos
1	Ana Silva	1
2	Carlos Souza	1
3	Fernanda Lima	0
4	Lucas Mendes	0

Expectativa: Uma tabela listando todos os clientes e a respetiva contagem numérica de pedidos (com zeros para quem não comprou).

Interpretação: O uso de COUNT(p.id_pedido) (contando a coluna da tabela secundária em vez de COUNT(*)) faz com que linhas preservadas com NULL resultem corretamente em 0.

Pergunta 2: Qual é o valor monetário total gasto em pedidos por cada cliente, garantindo que os clientes inativos apareçam com total zero?

```
187 •  SELECT c.id_cliente, c.nome_cliente, COALESCE(SUM(ip.subtotal), 0) AS valor_total_gasto
188    FROM CLIENTE c
189    LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
190    LEFT JOIN ITEM_PEDIDO ip ON p.id_pedido = ip.id_pedido
191    GROUP BY c.id_cliente, c.nome_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

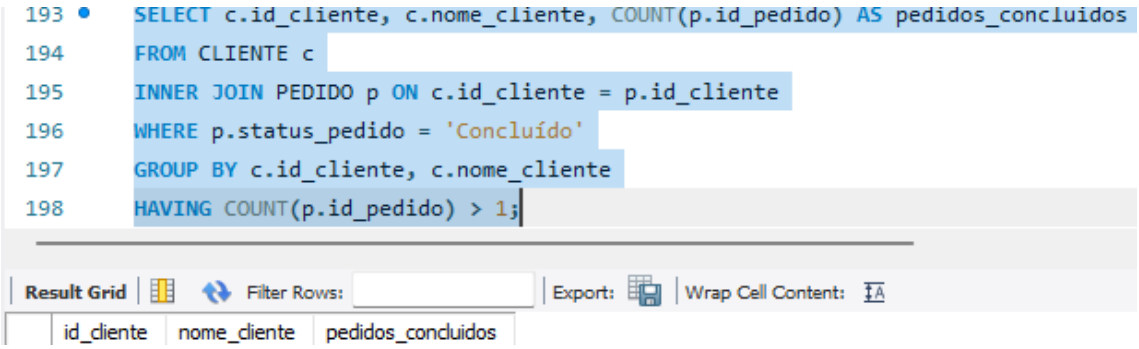
id_cliente	nome_cliente	valor_total_gasto
1	Ana Silva	150.00
2	Carlos Souza	161.00
3	Fernanda Lima	0.00
4	Lucas Mendes	0.00

Expectativa: Exibir o acumulado financeiro por cliente, tratando valores nulos como 0 para quem não gerou receita.

Interpretação: Encadeia junções externas e utiliza a função COALESCE para transformar eventuais valores nulos de somatórios em zero legível.

Uma consulta que combine WHERE e HAVING:

Pergunta: Quais são os clientes que realizaram mais de um pedido, considerando apenas os registos de pedidos que já se encontram com o status de "Concluído"?



Expectativa: Retornar apenas os clientes que ultrapassam o limite de um pedido concluído.

Interpretação: O WHERE filtra previamente as linhas individuais por status antes do agrupamento; em seguida, o HAVING analisa o resultado agregado do COUNT para restringir o grupo final.

Código Completo:

```
CREATE DATABASE IF NOT EXISTS loja_exercicio8;
USE loja_exercicio8;

CREATE TABLE CLIENTE(
id_cliente INT AUTO_INCREMENT PRIMARY KEY,
nome_cliente VARCHAR (50) NOT NULL,
telefone_cliente VARCHAR (11) NOT NULL,
email_cliente VARCHAR (100) UNIQUE,
CONSTRAINT chk_email_valido CHECK (
        email_cliente LIKE '%_@_%.%'
        AND CHAR_LENGTH(email_cliente)>=6)
);

-- 2. Tabela PRODUTO (independente)
CREATE TABLE PRODUTO (
    id_produto INT AUTO_INCREMENT PRIMARY KEY,
    nome_produto VARCHAR(50) NOT NULL,
    preco_produto DECIMAL(10,2) NOT NULL
```

```
);

-- 3. Tabela EMPREGADO (para o autorrelacionamento de supervisor)
CREATE TABLE EMPREGADO (
    id_empregado INT AUTO_INCREMENT PRIMARY KEY,
    nome_empregado VARCHAR(50) NOT NULL,
    id_supervisor INT,
    CONSTRAINT fk_supervisor FOREIGN KEY (id_supervisor) REFERENCES EMPREGADO(id_empregado)
);

-- 4. Tabela PEDIDO (depende de CLIENTE)
CREATE TABLE PEDIDO (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT NOT NULL,
    status_pedido VARCHAR(30) NOT NULL,
    CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente) REFERENCES CLIENTE(id_cliente)
);

-- 5. Tabela ITEM_PEDIDO (depende de PEDIDO e PRODUTO)
CREATE TABLE ITEM_PEDIDO (
    id_item INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade INT NOT NULL,
    subtotal DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido) REFERENCES PEDIDO(id_pedido),
    CONSTRAINT fk_item_produto FOREIGN KEY (id_produto) REFERENCES PRODUTO(id_produto)
);

-- 1. Inserir Clientes
```

```
INSERT INTO CLIENTE (nome_cliente, telefone_cliente, email_cliente) VALUES
('Ana Silva', '11988887777', 'ana@email.com'),
('Carlos Souza', '11977776666', 'carlos@email.com');

-- 2. Inserir Produtos
INSERT INTO PRODUTO (nome_produto, preco_produto) VALUES
('Teclado Mecânico', 150.00),
('Mouse Gamer', 80.50);

-- 3. Inserir Empregados (Autorrelacionamento: o chefe vem primeiro ou é NULL)
INSERT INTO EMPREGADO (nome_empregado, id_supervisor) VALUES
('João Gerente', NULL),            -- Chefe (sem supervisor)
('Maria Funcionaria', 1);          -- Subordinada (supervisor é o João, id 1)

-- 4. Inserir Pedidos (Vinculado aos clientes cadastrados)
INSERT INTO PEDIDO (id_cliente, status_pedido) VALUES
(1, 'Em Andamento'),
(2, 'Concluído');

-- 5. Inserir Itens do Pedido (Vinculado a pedidos e produtos)
INSERT INTO ITEM_PEDIDO (id_pedido, id_produto, quantidade, subtotal) VALUES
(1, 1, 1, 150.00), -- Pedido 1, Produto 1, Qtd 1
(2, 2, 2, 161.00); -- Pedido 2, Produto 2, Qtd 2

-- 1. Novos Clientes (Estes NÃO terão pedidos associados, ideais para testar o LEFT JOIN)
INSERT INTO CLIENTE (nome_cliente, telefone_cliente, email_cliente) VALUES
('Fernanda Lima', '11966665555', 'fernanda@email.com'),
('Lucas Mendes', '11955554444', 'lucas@email.com');

-- 2. Novos Produtos (Estes NÃO serão incluídos em nenhum item de pedido, ideais para testar produtos nunca vendidos)
INSERT INTO PRODUTO (nome_produto, preco_produto) VALUES
```

```
('Monitor Ultrawide', 1200.00),
('Webcam Full HD', 250.00);

SELECT p.id_pedido, c.nome_cliente, p.status_pedido
FROM PEDIDO p
INNER JOIN CLIENTE c ON p.id_cliente = c.id_cliente;

SELECT pr.nome_produto, ip.subtotal
FROM ITEM_PEDIDO ip
INNER JOIN PRODUTO pr ON ip.id_produto = pr.id_produto;

-- p = pedido / pr = produto / ip = item_pedido
SELECT p.id_pedido , pr.nome_produto , ip.quantidade , ip.subtotal
FROM PEDIDO p
INNER JOIN ITEM_PEDIDO ip ON p.id_pedido = ip.id_pedido
INNER JOIN PRODUTO pr ON ip.id_produto = pr.id_produto

-- e = empregado / s = supervisor
SELECT e.nome_empregado AS Empregado , s.nome_empregado AS Supervisor
FROM EMPREGADO e
INNER JOIN EMPREGADO s ON e.id_supervisor = s.id_empregado;

SELECT c.id_cliente, c.nome_cliente, p.id_pedido, p.status_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;

SELECT pr.id_produto, pr.nome_produto
FROM PRODUTO pr
LEFT JOIN ITEM_PEDIDO ip ON pr.id_produto = ip.id_produto
WHERE ip.id_item IS NULL;

SELECT c.nome_cliente, p.id_pedido
```

```
FROM CLIENTE c
RIGHT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;

-- 1ª Parte: Traz todos os clientes e os pedidos correspondentes (ou NULL se não tiverem pedidos)
SELECT c.id_cliente, p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente

UNION -- Junta os resultados e remove duplicados

-- 2ª Parte: Traz todos os pedidos e os clientes correspondentes (ou NULL se não tiverem cliente)
SELECT c.id_cliente, p.id_pedido
FROM CLIENTE c
RIGHT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;

ALTER TABLE PRODUTO ADD categoria_produto VARCHAR(30);

SELECT
    categoria_produto,
    COUNT(*) AS quantidade_produtos,
    MIN(preco_produto) AS menor_preco,
    MAX(preco_produto) AS maior_preco,
    AVG(preco_produto) AS media_preco
FROM PRODUTO
GROUP BY categoria_produto;

SELECT
    c.id_cliente,
    c.nome_cliente,
    COUNT(p.id_pedido) AS total_pedidos
FROM CLIENTE c
```

```
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome_cliente;

SELECT
    c.id_cliente,
    c.nome_cliente,
    COUNT(p.id_pedido) AS total_pedidos
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome_cliente
HAVING COUNT(p.id_pedido) >= 2;

SELECT
    id_pedido,
    SUM(subtotal) AS valor_total_pedido
FROM ITEM_PEDIDO
GROUP BY id_pedido;

SELECT c.nome_cliente, p.status_pedido
FROM CLIENTE c
INNER JOIN PEDIDO p ON c.id_cliente = p.id_cliente;

SELECT p.nome_produto, ip.quantidade, ip.subtotal
FROM ITEM_PEDIDO ip
INNER JOIN PRODUTO p ON ip.id_produto = p.id_produto;

SELECT c.nome_cliente, p.id_pedido, p.status_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente;

SELECT p.id_produto, p.nome_produto
FROM PRODUTO p
```

```
LEFT JOIN ITEM_PEDIDO ip ON p.id_produto = ip.id_produto
WHERE ip.id_item IS NULL;

SELECT e.nome_empregado AS Empregado, s.nome_empregado AS Supervisor
FROM EMPREGADO e
INNER JOIN EMPREGADO s ON e.id_supervisor = s.id_empregado;

SELECT c.id_cliente, c.nome_cliente, COUNT(p.id_pedido) AS total_pedidos
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome_cliente;

SELECT c.id_cliente, c.nome_cliente, COALESCE(SUM(ip.subtotal), 0) AS
valor_total_gasto
FROM CLIENTE c
LEFT JOIN PEDIDO p ON c.id_cliente = p.id_cliente
LEFT JOIN ITEM_PEDIDO ip ON p.id_pedido = ip.id_pedido
GROUP BY c.id_cliente, c.nome_cliente;

SELECT c.id_cliente, c.nome_cliente, COUNT(p.id_pedido) AS pedidos_concluidos
FROM CLIENTE c
INNER JOIN PEDIDO p ON c.id_cliente = p.id_cliente
WHERE p.status_pedido = 'Concluído'
GROUP BY c.id_cliente, c.nome_cliente
HAVING COUNT(p.id_pedido) > 2;
```